

中山大學

SUN YAT-SEN UNIVERSITY

# 实验 0x05 实验报告

## fork 的实现、并发与锁机制


姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院


2025 学年第二学期

# 1. fork 的实现

## 1.1. 系统调用

首先, 我们设置系统调用号为 `58`, 将用户态库中给出的系统调用对接到内核对应的函数当中，部分代码如下

```
#[inline(always)]
pub fn sys_fork() -> u16 {
    syscall!(Syscall::Fork) as u16
}

Syscall::Fork => {
    crate::proc::fork(context);
}
```

## 1.2. 进程管理

首先，我们实现最外层的 `fork`

```
pub fn fork(context: &mut ProcessContext) {
    x86_64::instructions::interrupts::without_interrupts(|| {
        let manager = get_process_manager();
        // FIXED: save_current as parent
        manager.save_current(context);
        // FIXED: fork to get child
        manager.fork();
        // FIXED: push to child & parent to ready queue
        // FIXED: switch to next process
        manager.switch_next(context);
    })
}
```

具体而言，就是保存当前上下文，然后调用 `ProcessManager::fork` 委托给下一层处理，然后再切换到下一进程的上下文，这里的将子进程放入就绪队列的操作在下层 `fork` 实现。

然后，实现 `ProcessManager::fork`

```rust
pub fn fork(&self) {
    // FIXED: get current process
    let current = self.current();
    // FIXED: fork to get child
    let child = current.fork();
    let child_pid = child.pid();
    // FIXED: add child to process list
    self.add_proc(child_pid, child.clone());
    self.push_ready(child_pid);
    // FOR DBG: maybe print the process ready queue?
    log::trace!(
        "Process {:?} successfully forked child {:?}",
        current.pid(),
        child_pid
    );
}
```

这里是将当前的 `fork` 任务委托到下一层进程自己的 `Process::fork` 执行，然后本层要做的就是把子进程放入就绪队列并加入进程管理表中。

接着实现 `Process::fork`

```rust
pub fn fork(self: &Arc<Self>) -> Arc<Self> {
    // FIXED: lock inner as write
    let mut inner = self.write();
    // FIXED: inner fork with parent weak ref
    let child_inner = inner.fork(Arc::downgrade(self));

    // FOR DBG: maybe print the child process info
    //          e.g. parent, name, pid, etc.
    log::trace!(
        "Process {}#{} forked a child process with pid: {}",
        inner.name(),
        self.pid,
        child_inner.parent().unwrap().pid()
    );
    // FIXED: make the arc of child
    let child = Arc::new(Self {
        pid: ProcessId::new(),
        inner: spin::RwLock::new(child_inner),
    });
    // FIXED: add child to current process's children list
    inner.children.push(child.clone());
    // FIXED: set fork ret value for parent with `context.set_rax`
    inner.context.set_rax(child.pid().0 as usize);
    // FIXED: mark the child as ready & return it
    child
}
```

这一层需要做的是，委托到 `ProcessInner::fork`，然后把当前获得的子进程的 `ProcessInner` 和其 `pid` 打包，并放入当前进程的子进程表中，并设置父进程的 `rax` 返回值寄存器为子进程的 `pid`。

然后实现的是 `ProcessInner::fork`

```
pub fn fork(&mut self, parent: Weak<Process>) -> ProcessInner {
        // FIXED: fork the process virtual memory struct
        // FIXED: calculate the real stack offset
        let stack_offset_count = self.children.len() as u64 + 1;
        let child_vm = self.vm().fork(stack_offset_count);
        // FIXED: clone the process data struct
        let mut child_context = self.context.clone();

        // FIXED: update `rsp` in interrupt stack frame
        // FIXED: set the return value 0 for child with `context.set_rax`
        let stack_offset = stack_offset_count * STACK_MAX_SIZE;

        child_context.as_mut().stack_frame.stack_pointer -= stack_offset;
        child_context.as_mut().regs.rbp -= stack_offset as usize;
        child_context.set_rax(0);
        // FIXED: construct the child process inner
        let child_data = self.proc_data.clone();
        // NOTE: return inner because there's no pid record in inner
        ProcessInner {
            name: self.name.clone(),
            parent: Some(parent),
            children: Vec::new(),
            ticks_passed: 0,
            status: ProgramStatus::Ready,
            context: child_context,
            exit_code: None,
            proc_data: child_data,
            proc_vm: Some(child_vm),
        }
    }
}
```

这一层需要做的是，委托到内存管理器，建立对应的页表映射和栈，然后本层做的是计算出对应的栈指针地址，这里需要 `+1` 是因为上层 `fork` 是先调用了当前的 `fork` 之后再把子进程插入子进程 `Vec` 中，建立了对应的页表映射之后需要修改栈指针，设置子进程的 `rax` 返回值寄存器的值为 `0`，然后构造出子进程的 PCB 并返回。

最后要实现的就是栈的建立和复制了，如下

```rust
pub fn fork(
    &self,
    mapper: MapperRef,
    alloc: FrameAllocatorRef,
    stack_offset_count: u64,
) -> Self {
    // FIXED: alloc & map new stack for child (see instructions)
    let cur_stack_base = self.range.start.start_address().as_u64();
    let mut new_stack_base = cur_stack_base - stack_offset_count *
STACK_MAX_SIZE;

    // log::info!("1. current: {}, new: {}", cur_stack_base, new_stack_base);
    // log::info!("usage: {} range: {}", self.usage, cur_stack_base);

    while elf::map_range(new_stack_base, self.usage, mapper, alloc).is_err() {
        trace!("Map thread stack to {:#x} failed.", new_stack_base);
        new_stack_base -= STACK_MAX_SIZE; // stack grow down
    }
    // log::info!("2. current: {}, new: {}", cur_stack_base, new_stack_base);
    // FIXED: copy the *entire stack* from parent to child
    self.clone_range(cur_stack_base, new_stack_base, self.usage);
    // FIXED: return the new stack
    Self {
        range: Page::range(
            Page::containing_address(VirtAddr::new(new_stack_base)),
            Page::containing_address(VirtAddr::new(new_stack_base)) + self.usage,
        ),
        usage: self.usage,
    }
}
```

具体而言，就是先计算出子进程的栈的栈底，然后建立页表映射，最后复制父进程的数据，把栈的 `range` 和 `usage` 信息打包后返回。

然后尝试运行 `fork_test` 进行测试，

```
[>] fork_test
I am the child process
I am the parent process
  PID | PPID | Process Name |  Ticks   | Status | Memory Usage
 #  3 | #  2 | fork_test    |        4 | Running | 28672 B (7 Pages)
 #  2 | #  1 | sh           |    13201 | Ready | 28672 B (7 Pages)
 #  4 | #  3 | fork_test    |        3 | Ready | 28672 B (7 Pages)
 #  1 | #  0 | kernel       |    13216 | Ready | 2097152 B (512 Pages)
Queue  : [4, 2, 1]
CPUs   : [0: 3]
Waiting for child to exit...
child read value of M: 0xdeadbeef
child changed the value of M: 0x2333
Child exited with status 64
parent read value of M: 0x2333
Process [3] exited with code: 1056
```

可以看到，`fork_test` 正常运行并退出，可知 `fork` 的实现大致无误。

## 2. 进程的阻塞与唤醒

这个部分的实现比较简单，主要就是把 `wait_pid` 从轮询的方式改为阻塞，每次将一个需要等待的进程放入被等待进程的等待队列中然后阻塞（设置为 `blocked` 状态）。当那个被等待的进程被 `kill` 了之后，把所有等待它的进程唤醒，将状态改为 `ready` 并放入就绪队列。

实现了基于阻塞的 `wait_pid` 后进行测试，可以看到原本的 `sh` 被阻塞，当当前的 `sh` 退出后，又调度回到 `running` 态，可知这部分功能实现大致无误。

```
Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>] sh
Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>] stat
  PID | PPID | Process Name |  Ticks   | Status | Memory Usage
 #  3 | #  2 | sh           |   14418 | Running | 28672 B (7 Pages)
 #  1 | #  0 | kernel       |   22969 | Ready | 2097152 B (512 Pages)
 #  2 | #  1 | sh           |    8542 | Blocked | 28672 B (7 Pages)
Queue  : [1]
CPUs   : [0: 3]
[>] exit
Goodbye!
Process [3] exited with code: 0
[>] stat
  PID | PPID | Process Name |  Ticks   | Status | Memory Usage
 #  1 | #  0 | kernel       |  225334 | Ready | 2097152 B (512 Pages)
 #  2 | #  1 | sh           |   17514 | Running | 28672 B (7 Pages)
Queue  : [1]
CPUs   : [0: 2]
[>] exit
Goodbye!
[INFO] ysos_kernel - YatSenOS shutting down.
```